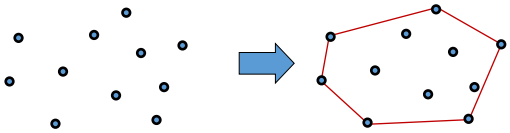
წერტილების სიმრავლის ამოზნექილი გარსი (Convex hull) 10.4

ამოცანა ძირითადად განიხილება 2D და 3D სივრცეებისთვის. აქ მოყვანილია ოთხი ალგორითმი 2D შემთხვევისთვის.

მოცემულია: N წერტილი თავისი კოორდინატებით x,y.

ამოცანა: ასაგებია ამოზნექილი მინიმალური ფართის გარსი, რომელიც შეიცავს ყველა მოცემულ წერტილს



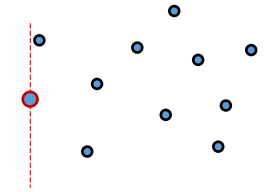
ჯარვისის ალგორითმი (საჩუქრის შეფუთვის მეთოდი) (Gift wrapping algorithm; Jarvis march) 10.4.1

ჯარვისის ალგორითმის ინტუიტიური წარმოდგენა: წერტილების სიმრავლეს გარედან ვახვევთ შესაფუთ ქაღალდს:

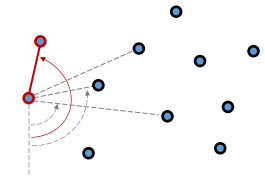
ალგორითმის მსვლელობა:

1. ფიქსირდება წერტილი, რომლის x ან y კოორდინატა მინიმალურია.

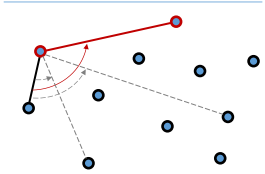
ბუნებრივია, რომ წერტილი მინიმალური (ან მაქსიმალური) კოორდინატის მნიშვნელობით გარსზე მდებარეობს. ანუ გარსის პირველი წერტილი უკვე მოძიებულია.



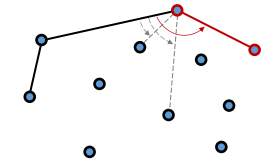
2. ამ წერტილიდან იღება კუთხეები (-y)მიმართულებასა და სხვა წერტილებს შორის. წერტილი, რომელიც იძლევა ყველაზე დიდ კუთხეს, ასევე იქნება გარსზე.



3. გადასვლა ამ მეორე წერტილზე და აღება ყველა კუთხის წინა (ამ შემთხვევაში პირველი) წერტილისა და სხვა წერტილებზე მიმართულებებს შორის. ისევ ფიქსირდება მაქსიმალური კუთხე (კუთხეები იქნება $\alpha \leq 180°$). რომელიც შეესაბამება შემდეგ წერტილს გარსზე.

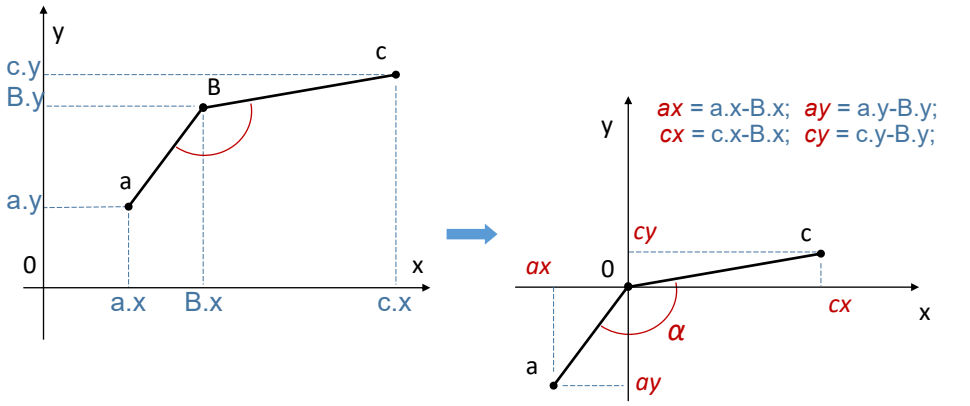


4. პროცედურა მეორდება მანამ, სანამ არ მივა საწყის წერტილამდე



ალგორითმის დასაპროგრამებლად დაგვჭირდება ფუნქცია, რომელიც გამოთვლის კუთხეს სამ წერტილს შორის: წინა წერტილი - მიმდინარე წერტილი - ახალი წერტილი.

სიმარტივისთვის დავამთხვიოთ B წერტილი ნოლს და დავთვალოთ კუთხე კოორდინატთა სათავედან გამოსულ ორ ვექტორს შორის



$$\alpha = \frac{ax * cx + ay * cy}{\sqrt{(ax^2 + ay^2) * (cx^2 + cy^2)}}$$

ქვემოდმოყვანილი საილუსტრაციო პროგრამას გამოყავს ეკრანზე მოცემული რაოდენობის წერტილი. თითო წერტილს ენიჭება უნიკალური ნომერი და ფერი. წერტილი იხაზება პატარა კვადრატების სახით. შემდეგ ხდება ჯარვისის ალგორითმის მეშვეობით გარსის დადგენა და გამოხაზვა. გარსის თითო მონაკვეთი იხაზება იმ ფერში, რა ფერის არის შემდეგი დადგენილი წერტილი.

ასევე პროგრამა წერს ტექსტურ ფანჯარაში თითო კვანძისთვის იმ კუთხის მნიშვნელობას, რომელმაც მასთან „მიიყვანა“ (ანუ წინა კვანძთან მიღებული გარსის კუთხე).

გამოყენებული მონაცემთა ტიპები:

```
struct feri { int r; int g; int b; };      // ფერი RGB ფორმატში
struct wertili {  double x; double y;  }; // 2D წერტილი

// კვანძი (კოორდინატები + კვანძის ნომერი + კვანძის ფერი )
struct kvandzi { wertili coord; int number; feri col; };
```

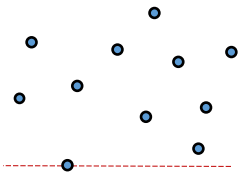
```
// დამხმარე ფუნქცია წერტილის ადგილზე კვადრატის დასმისთვის:

int draw_quadrate(int x, int y, int rad, int col_r, int col_g, int col_b )
{   setfillstyle(SOLID_FILL, COLOR(col_r,col_g,col_b)); // ფერის დაყენება
    bar (x-rad,y-rad, x+rad, y+rad); // კვადრატი ცენტრით x,y - ში
}

// სამ წერტილის კუთხის გაანგარიშების ფუნქცია (განხილულია წინა გვერდზე)

double angle_aBc(kvandzi a, kvandzi B, kvandzi c)
{
  double ax = a.coord.x - B.coord.x, ay = a.coord.y - B.coord.y;
  double cx = c.coord.x - B.coord.x, cy = c.coord.y - B.coord.y;

  double cosinuss = (ax*cx+ay*cy)/(sqrt((ax*ax+ay*ay))*sqrt(cx*cx+cy*cy));
  return acos(cosinuss)*180/3.14159; // რადიანების გრადუსებში გადაყვანა
 }


int main()
{
 initwindow(1000,700);          // გრაფიკული ფანჯრის ზომები
 setbkcolor(COLOR(20,20,20)); // ფონის ფერი
 cleardevice();
 char str[4];
 int i,j,min, imin;
 wertili punto;
 int TOTAL = 60;               // წერტილების რაოდენობა

 kvandzi uzel[TOTAL];          // წერტილების მასივის გამოცხადება

 srand(3);                     // შემთხვევითი რიცხვების გენერატორის დაყენება

 for (i = 0; i<TOTAL; i++)     // ამ ციკლში ვავსებთ მასივს წერტილებით
  {
      punto.x = rand()%300+rand()%300+50;// ორი შემთხვევითი რიცხვის ჯამი გვაძლევს
      punto.y = rand()%250+rand()%250+80;// უფრო მაღალ კონცენტრაციას შუაში.
                                         // ეს ბლოკი მოგვცემს წერეტილების
                                         //  კოორდინატებს დიაპაზონში [50-650]x
                                         // [80-580]

      uzel[i].coord = punto;         // მასივის ელემენტს ენიჭება წერტილის კოორდ.
      uzel[i].number = i;            // წერტილის ნომერი (ემთხვევა ელემენტის ნომერს)

      uzel[i].col.r = rand()%255;    // ფერის სამივე კომპონენტი - შემთხვევითი რიცხვი
      uzel[i].col.g = rand()%255;    // 0-დან 255-მდე
      uzel[i].col.b = rand()%255;
  }  // წერტილების მასივი მზადაა. ახლა დავიწყებთ ჯარვისის ალგორითმის აღწერას

 int leftest_index = 0;
 min = uzel[0].coord.x;
 for( i = 1; i<TOTAL; i++)          // ამ ციკლში ვპოულობთ ელემენტს მინიმალური
       if(uzel[i].coord.x  < min )  // x - კოორდინატით
          { min = uzel[i].coord.x; leftest_index = i; }
```

```
for( i=0; i<TOTAL; i++)   // ამ ციკლში გამოგყავს ეკრანზე წერტილები (მართკუთხედები)
   {
      settextstyle(2, HORIZ_DIR, 4);
      setcolor(COLOR(uzel[i].col.r, uzel[i].col.g, uzel[i].col.b));
      draw_quadrate(uzel[i].coord.x, uzel[i].coord.y, 3, uzel[i].col.r, uzel
[i].col.g, uzel[i].col.b  );
     sprintf(str,"%d", i );
     outtextxy(uzel[i].coord.x, uzel[i].coord.y, str ); // წერტილების ნომრები
   }

double max_angle;
int max_angle_index = 0;
double current_angle;

////
int morigi = leftest_index;
kvandzi previos = uzel[leftest_index];  // პირველი წერტილის წინ ვაყენებთ
previos.coord.y =  previos.coord.y+100; // "ვირტუალურ წერტილს" (-y)
                                        // მიმართულებიდან
do          // მთავარი ციკლი:
{
   max_angle = -10; max_angle_index = 0;
   for (int ii = 0; ii< TOTAL; ii++)      // ყველა წერტილს ვსინჯავთ
     {
      current_angle = angle_aBc(previos, uzel[morigi], uzel[ii]  );
      if ( current_angle > max_angle)     // ვეძებთ მაქსიმალურ კუთხეს
           {
             max_angle = current_angle;
             max_angle_index = ii;        // ii - შემდეგი გარსის წერტილის ნომერი
            }
     }
   printf ("\n num: %d : angle: %g",max_angle_index, max_angle ); // ვბეჭდავთ

   setcolor(COLOR(uzel[max_angle_index].col.r, uzel[max_angle_index].col.g, uzel
[max_angle_index].col.b));
   line(uzel[morigi].coord.x, uzel[morigi].coord.y, uzel
[max_angle_index].coord.x, uzel[max_angle_index].coord.y ); // ხაზის გამოხაზვა
   previos = uzel[morigi];  // მორიგი ელემენტი ხდება previos (წინა)
   morigi = max_angle_index; // მორიგი კვანძის ნომერი ხდება ახალი წერტილის ნომერი
}
while (morigi != leftest_index); // სანამ არ დავბრუნდებით პირველ წერტილში

getch();
closegraph();
}
```

ჯარვისის ალგორითმის შრომატევადობაა ***O(Nh)***, სადაც ***N*** - წერტილების, ხოლო ***h***- გარსის წიბოების რაოდენობაა

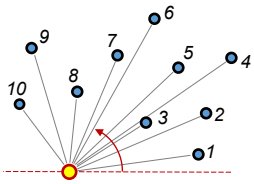
10.4.2 გრეჰემის ალგორითმი (Graham scan)

გრეჰემის მეთოდის ერთადერთი მსგავსება ჯარვისის ალგორითმთან იმაშია, რომ აქაც პირველ რიგში იძებნება წერტილი მინიმალური კოორდინატით.



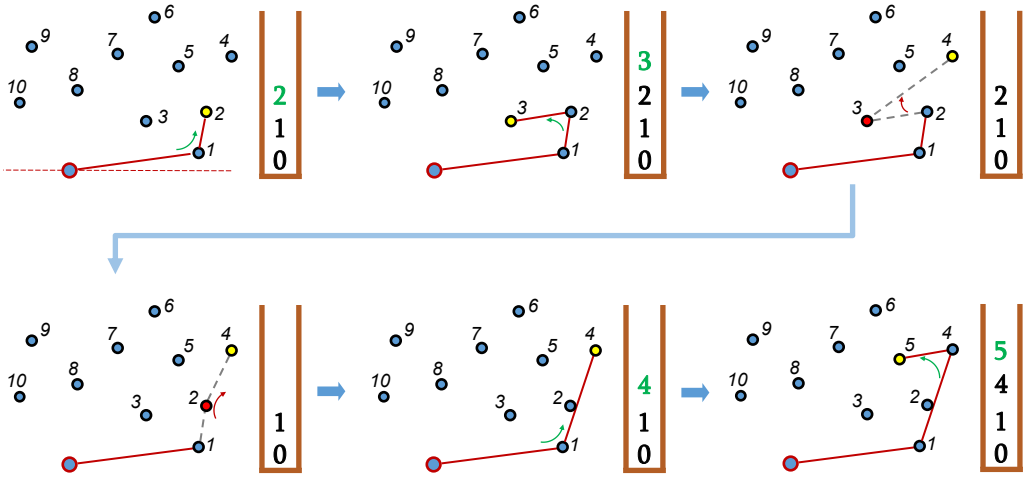
იღება, მაგალითად, საწყის წერტილად მინიმალური y კოორდინატის მქონე წერტილი.

შემდეგ ყველა სხვა წერტილი ლაგდება მათი საწყის წერტილან მიმართებით პოლარული კოორდინატების მიხედვით (ანუ კუთხის მიხედვით, და თუ კუთხე ერთნაირია—დაშორების მიხედვით). (სურ:) წერტილების სორტირებას დასჭირდება *O(N*log(N))* ოპერაცია.



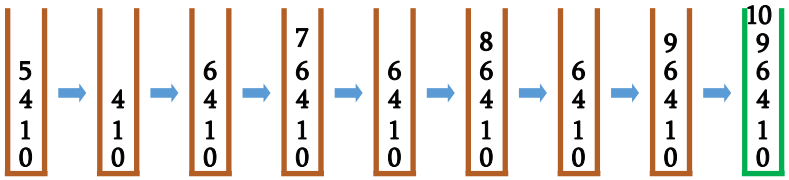
გრეჰემის ალგორითმისთვის მიზანშეწონილია სტეკის გამოყენება, რომელშიც შეინახება წერტილების ნომრები. სტეკის ფუნქციებს შორის საჭიროა ისეთიც, რომელიც წაიკითხავს სტეკის ზედა ელემენტის მნიშვნელობას, მისი სტეკიდან ამოღების გარეშე. თავიდან სტეკში იწერება 0 და 1 წერტილების ნომრები (სურ.)

შემდეგ დიდი ციკლით **`for (i=2; i<N; i++)`** ხდება წერტილების მთელი მასივის შემოვლა. ამ დიდი ციკლის შიგნით დგინდება, იძლევა თუ არა ახალი წერტილი მარცხენა ბრუნვას. თუ არა, სტეკიდან წერტილები ამოიღება მანამ, სანამ არ მიიღება მარცხენა ბრუნვა. შემდეგ ახალი წერტილი იწერება სტეკში.



მწვანე ისარი (მარცხენა ბრუნვა) - სტეკს ემატება ელემენტი,
წითელი ისარი (მარჯვენა ბრუნვა) - სტეკი ელემენტს კარგავს

და ასე გრძელდება მანამ, სანამ არ იქნება გავლილი წერტილებს მასივი.
სურათზე მოცემული მაგალითისთვის სტეკის მომავალი მდგომარეობები იქნება:



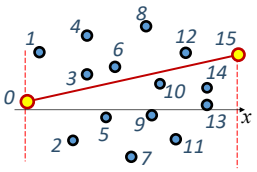
დიდი ციკლის შესრულების შემდეგ სტეკში დარჩება იმ წერტილების ჩამონათვალი, რომლებზეც გადის ამოზნექილი გარსი

ალგორითმის ფსევდოკოდი. თავიდან - ტიპები და ფუნქციები:

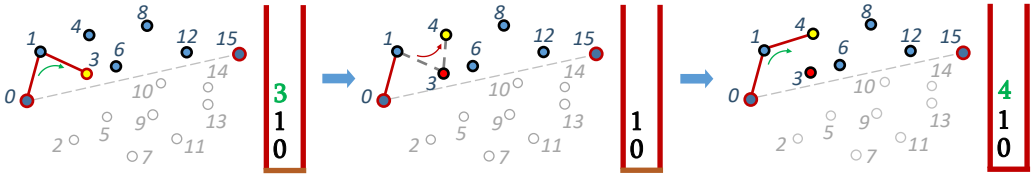
```
stack S -       // S იყოს სტეკი, რომელშიც შევინახავთ წერტილების ნომრებს;
                // სტეკს უნდა ჰქონდეს ორი სპეციფიური ფუნქცია:
                // GetTop(S) - რომელიც წაიკითხავს სტეკის ზედა ელემენტს,
                // მაგრამ არ შეცვლის სტეკის შიგთავსს (არ წაშლის სტეკიდან)
                // GetSubTop(S) - რომელიც წაიკითხავს სტეკიდან ზევიდან მეორე
                // ელემენტს და ასევე არ შეცვლის მას.

boolean LeftTurn (point A, point B, point C) //- ფუნქცია, რომელიც
                                    // აბრუნებს true - ს, თუ არგუმენტის სამი
                                    // წერტილის შემოვლა გვაძლევს მარცხენა
                                    // მობრუნებას

/* P[0] - იყოს წერტილი მინიმალური y კოორდინატით (თუ ასეთი წერტილი
მეტია, მაშინ მათ შორის—მინიმალური x კოორდინატით). (იხ. ჯარვისის
ალგორითმის პროგრამის შესაბამისი ნაწილი) */

polar_sort (P[0], P);          // ვასორტირებთ სხვა წერტილებს (P[1]..P[N])
                               // იმის მიხედვით, თუ რა პოლარული კუთხე
                               // აქვთ მათ P[0] წერტილიდან OX ღერძის მიმართ.
push(0, S); push( 1, S);       // პირველი ორი წერტილის ნომრები იწერება სტეკში;
                               //  ეს ორი წერტილი გარანტირებულად იქნება გარსზე.

for (i = 2; i<N; i++)          // მთავარი ციკლი
 {
        while ( LeftTurn (P[GetSubTop(S) ], P[GetTop(S)], P[i]) == false) )
        {               // სანამ სტეკის ბოლო ორი წერტილი და მორიგი წერტილი
                        // არ გვაძლევს მარცხენა კუთხეს...
           pop (S);     // მაშინ სტეკიდან ვაშორებთ ბოლო წერტილის ნომერს
         }
        push(i, S);     // სტეკში ვამატებთ ახალ ელემენტ i-ს
  }
 return (S);
```

ენდრიუს ალგორითმი გრეჰემის ალგორითმის ვარიაციაა. ოღონდ აქ წერტილების ნუმერაცია ხდება არა პოლარული კუთხის , არამეტ მათი x - კოორდინატის მიხედვით, რაც უფრო სწრაფია.

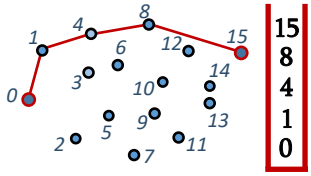
ალგორითმის მსვლელობა: თავიდან წერტილების მასივი სორტირდება ლექსიკოგრაფიულად x - კოორდინატის (და x - კოორდინატის ტოლობის შემთხვევაში, y-კოორდინატის) მიხედვით, ისე რომ პირველი წერტილი იყოს მინიმალური x -ით, ხოლო ბოლო - მაქსიმალურით. ამ ორ წერტილს შორის ივლება ხაზი (სურ.).



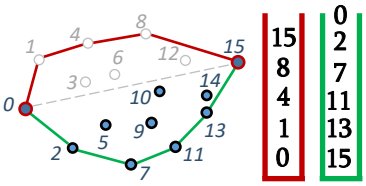
ალგორითმი იწყება პირველი წერტილიდან და მოძრაობს ბოლო წერტილამდე ისევე, როგორც გრეჰემის მეთოდის შემთხვევაში (სტეკიდან იშლება ბოლო ელემენტი ვექტორიც მარცხენა მობრუნების შემთხვევაში). *შეიძლება ალგორითმის მოდიფიცირება ისე, რომ მან შეამოწმოს მხოლოდ ის წერტილები, რომლებიც პირველ და ბოლო წერტილის დამაკავშირებელი ხაზის ზემოთ არიან.*
ასე ივსება გარსის ზედა კონტური. გარსის ქვედა კონტურს ექნება თავისი, მეორე, სტეკი და იქ ალგორითმი იწყება ბოლო წერტილიდან და მოდის პირველ წერტილამდე. შემდეგ ორივე სტეკის შინაარსი ერთიანდება.



პირველი წერტილიდან ბოლო წერტილამდე გავლის შედეგად მიიღება გარსის ზედა ნახევარი (სურ:)

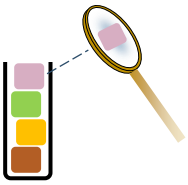


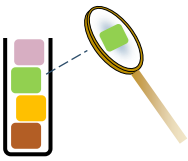
შემდეგ იწყება მეორე ციკლი, ბოლო წერტილიდან პირველამდე. (შეიძლება მხოლოდ ხაზის ქვემოთ მდებარე წერტილებს განხილვა). წერტილების ნომრები ინახება მეორე სტეკში).
სტეკების შიგთავსის გაერთიანება მარტივად ხდება *O (N)* დროში.

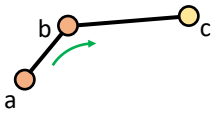


როგორც გრეჰემის, ასევე ანდერსენის მეთოდის როგორც ასეთის, სირთულეა წრფივი *(O (N))*, მაგრამ საჭიროა წერტილების წინასწარი სორტირება, რაც, სორტირების სწრაფი მეთოდების გამოყენებით, მოითხოვს *O(N*Log(N))* შრომატევადობას. ანუ ჯამში მაინც ***O(N*Log(N))*** სირთულე გამოდის.

```
// ქვემოთ მოყვანილია ანდერსენის ალგორითმის პროგრამის ფრაგმენტები. გამოტოვებულია აღწერა
// სტეკთან მომუშავე რამდენიმე სტანდარტული ფუნქციისა და შერწყმით დალაგების
// ფუნქციის, რომლის პარამეტრებში დამატებულია მითითება, თუ რომელი კოორდინატის მიხედვით
// დაალაგოს წერტილების მასივი.


const int MAX_LEN = 120;        // წერტილების რაოდენობა
struct node_2d                  // წერტილის კოორდინატები და ნომერი
  {
      double x;
      double y;
      int ID;
  };
struct hull_stack               // სტეკში შეინახება წერტილების ნომრები
  {
      int A[MAX_LEN];
      int top_index;
  };
//////// სტეკის ფუნქციები ( აქ მოყვანილია მხოლოდ ორი):
int pop_in_stack(hull_stack s)      // სტეკის ბოლო ელემენტის ნახვა
  {                                 // სტეკში ცვლილების შეტანის გარეშე
     if (s.top_index == -1) return 0;
      return s.A[s.top_index];
  }

int pop_2nd_in_stack(hull_stack s)  // სტეკის ბოლოდან მეორე ელემენტის
 {                                  // ნახვა სტეკში ცვლილების შეტანის გარეშე
     if (s.top_index < 1) return 0;
      return s.A[s.top_index-1];
  }

// a, b და c წერტილების შემოვლის კუთხე
// ( თუ s>0 - მარჯვენა, თუ s<0 - მარცხენა კუთხეა)
int right_turn(node_2d a, node_2d b, node_2d c)
{
      double s = (c.y-a.y)*(b.x-a.x) - (b.y-a.y)*(c.x-a.x);
      if(s==0) return 0;
      if(s>0) return 1;
      return -1;
}

//////////////////////////// მთავარი ფუნქცია:
main()
{
      int i;
      node_2d tmp;
      node_2d Point[MAX_LEN];       // წერტილების მასივი
      hull_stack st_1, st_2;        // გამოცხადებულია ორი სტეკი

      srand(7);                     // ემთხვევითი რიცხვების გენერატორის დაყენება
      for(i=0; i<MAX_LEN; i++)
          {
            Point[i].ID = i;
            Point[i].x = rand()%7000/10.0+50;   // რიცხვები კოორდ. x:[50-750]
            Point[i].y = rand()%5000/10.0+50;   // და y:[50-550] დიაპაზონში
          }                                     // სიზუსტით 1 ნიშანი მძიმის მერე
```





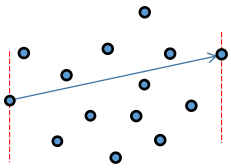


```
      mergesort (Point, 0, MAX_LEN-1, 2);  // 2: y-კოორდინატით სორტირება
      mergesort (Point, 0, MAX_LEN-1, 1);  // 1: x-კოორდინატით სორტირება

      create_stack(st_1);                        // შეიქმნა პირველი სტეკი
      push_to_stack(st_1, 0);                    // სტეკში ჩაიწერა პირველი
      push_to_stack(st_1, 1);                    // და მეორე ელემენტის ნომრები

      for (i=2; i<MAX_LEN; i++)           // მესამე ელემენტიდან იწყება შემოვლა:
          {                                                          // სანამ
              while (right_turn( Point[pop_2nd_in_stack(st_1)],  // სამი წერტილის
                                 Point[pop_in_stack(st_1)],      // შემოვლის კუთხე
                                 Point[i]) == -1 )               // მარცხენაა ...
                pull_from_stack(st_1);    //... სტეკს ვაშორებთ ბოლო ელემენტს
              push_to_stack(st_1, i);     // ახალი წვეროს (წერტილის) ნომერი - სტეკში!
            }
//// ამით დასრულდა ზედა კონტურის შემოვლა და პირველი სტეკის შევსება.
//// გადავდივართ ქვედა კონტურზე და მეორე სტეკის შევსებაზე:

      create_stack(st_2);                        // შეიქმნა მეორე სტეკი
      push_to_stack(st_2, MAX_LEN-1);            // სტეკში ჩაიწერა ბოლო
      push_to_stack(st_1, MAX_LEN-2);            // და ბოლოს წინა ელემენის ნომრები

      for (i=MAX_LEN-3; i>=0; i--)  // ბოლოდან მესამე ელემენტიდან იწყება შემოვლა:
          {                                                          // სანამ
              while (right_turn( Point[pop_2nd_in_stack(st_2)],  // სამი წერტილის
                                 Point[pop_in_stack(st_2)],      // შემოვლის კუთხე
                                 Point[i]) == -1)                // მარცხენაა...
                pull_from_stack(st_2);    //... სტეკს ვაშორებთ ბოლო ელემენტს
              push_to_stack(st_2, i);     // ახალი წვეროს (წერტილის) ნომერი - სტეკში!
            }

// ახლა ამოვწეროთ გარსის ვერტილების ნომრები სტეკებიდან:
      while( is_not_empty(st_1) ) // მანამ სანამ სტეკი არ დაცარიელდა:
       {
            tmp = Point[pull_from_stack(st_1)]; // ვიღებთ სტეკიდან ბოლო ელემენტს...
                                                // … და ვბეჭდავთ მას:
            printf("\nst_1: ID: %d, x: %g, y:%g ", tmp.ID, tmp.x, tmp.y );
       }

      // იგივეს ვიმეორებთ მეორე სტეკისთვის:
      while( is_not_empty(st_2) )
       {
            tmp = Point[pull_from_stack(st_2)];
            printf("\nst_2:ID: %d, x: %g, y:%g ", tmp.ID, tmp.x, tmp.y );
       }
    getch();
}

// ამ პროგრამაში არ არის გამოყენებული ალგორითმის აღწერაში მოყვანილი მასივის დაყოფა
// ორ ნაწილად:  ელემენტები, რომლებიც პირველი და ბოლო წვეროების დამაკავშირებელი
// ხაზის ზემოთ  და ამ ხაზის ქვემოთ ხვდებიან.
```

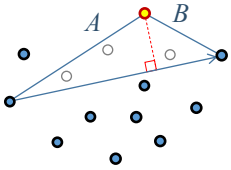
სწრაფი გარსის ალგორითმი (Quick hull) 10.4.4

ალგორითმის დასახელება Quick Hull (“სწრაფი გარსი”) მოდის ანალოგიიდან Quick sort - თან. და მართლაც, ამ ალგორითმებს შორის არის გარკვეული მსგავსება. ორივე იყენებს Divide and Conquer მეთოდს.

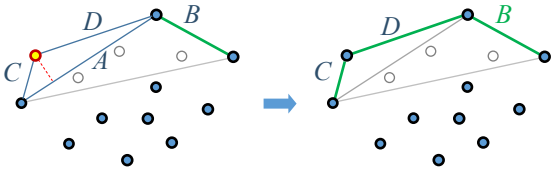
პირველ რიგში ირჩევა ორი წერტილი რომელიმე კოორდინატის (ჩვენ მაგალითში ავიღებთ x - ს) მინიმალური და მაქსიმალური მნიშვნელობით და დგინდება მათი დამაკავშირებელი ხაზი (სურ)



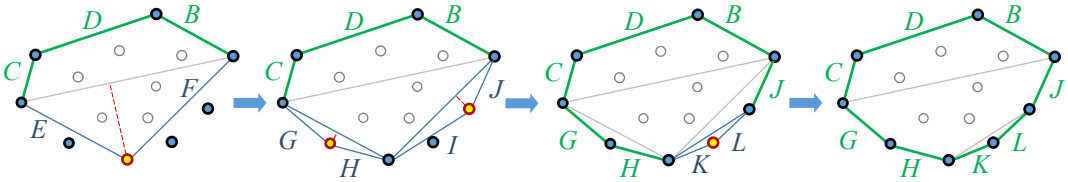
ეს ხაზი ყოფს წერტილების სიმრავლეს ორ ნაწილად: წერტილები, რომლებიც ხაზის ზემოთაა და ხაზის ქვემოთ. ჯერ განიხილება ერთი ნაწილი (მაგალითად, ზემოთ განლაგებული წერტილები და მათ შორის დგინდება ხაზიდან ყველაზე შორს მდებარე. ეს წერტილი უერთდება ხაზის საწყის და ბოლო წერტილებს ახალი ხაზებით *A* და *B*. ის წერტილები, რაც მოექცა სამკუთხედის შიგნით, შეგვიძლია გამოვრიცხოთ შემდგომი ანალიზიდან. (სურ)



შემდეგ ფუნქცია გამოიძახება რეკურსიულად A და B ხაზებისთვის. თუ ხაზის გარეთ არ დარჩა წერტილი, მაშინ ხაზი ხდება გარსის ნაწილი.



იგივე მეორდება წერტილების მეორე ნახევრისთვის:



ამ ალგორითმში ხდება *log(N)* დაყოფა და თითო დაყოფის დროს მუშავდება ***N*** წერტილი, ანუ საშუალოდ გამოდის ალგორითმის შრომატევადობა ***N*log(N)***, თუმცა, უარესს შემთხვევაში, როდესაც ყველა წერტილი გარსის ნაწილი გახდება, მისი შრომატევადობა იქნება $O(N^2)$